Multiple Regression on housing data

Ques :

For the house price prediction dataset. Implement Linear regression by considering all the features into account.

Report the coefficient of determination, MSE and the learning rate used.

Report whether the algorithm has reached the convergence or how many interations has been performed.

Report the final trained model

In [36]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

EDA

In [37]:
```
housing = pd.read_csv("Housepriceprediction.csv")
housing.head()
```

Out[37]:

	Id	LotArea	OverallQual	1stFlrSF	GrLivArea	TotRmsAbvGrd	GarageCars	GarageArea
0	1	8450	7	856	1710	8	2	548
1	2	9600	6	1262	1262	6	2	460
2	3	11250	7	920	1786	6	2	608
3	4	9550	7	961	1717	7	3	642
4	5	14260	8	1145	2198	9	3	836

In [38]:
```
housing.info()
```

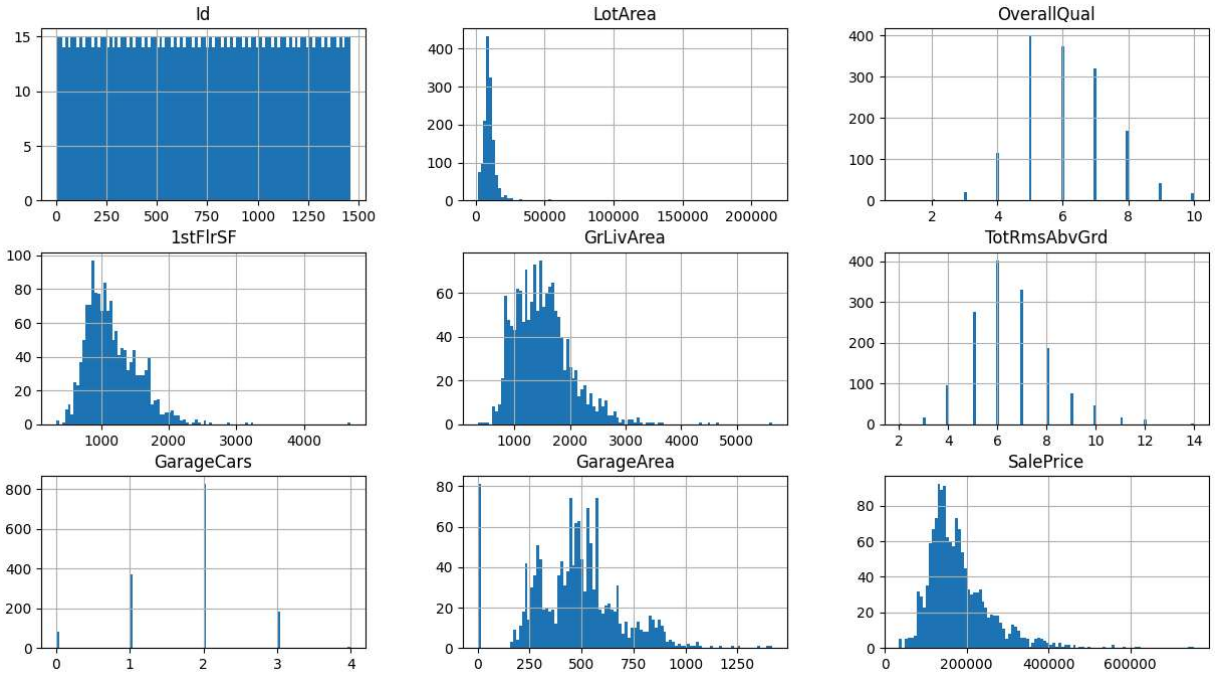
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Id            1460 non-null   int64
 1   LotArea       1460 non-null   int64
 2   OverallQual   1460 non-null   int64
 3   1stFlrSF      1460 non-null   int64
 4   GrLivArea     1460 non-null   int64
 5   TotRmsAbvGrd  1460 non-null   int64
 6   GarageCars    1460 non-null   int64
 7   GarageArea    1460 non-null   int64
 8   SalePrice     1460 non-null   int64
dtypes: int64(9)
memory usage: 102.8 KB
```

In [39]:
```
housing.describe()
```

Out[39]:

	Id	LotArea	OverallQual	1stFlrSF	GrLivArea	TotRmsAbvGrd
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000
mean	730.500000	10516.828082	6.099315	1162.626712	1515.463699	6.517808
std	421.610009	9981.264932	1.382997	386.587738	525.480383	1.625393
min	1.000000	1300.000000	1.000000	334.000000	334.000000	2.000000
25%	365.750000	7553.500000	5.000000	882.000000	1129.500000	5.000000
50%	730.500000	9478.500000	6.000000	1087.000000	1464.000000	6.000000
75%	1095.250000	11601.500000	7.000000	1391.250000	1776.750000	7.000000
max	1460.000000	215245.000000	10.000000	4692.000000	5642.000000	14.000000

In [40]:
```
housing.hist(bins=100, figsize=(15,8))
plt.show()
```



Regression Fit

1. All columns contain numerical data, which can be used.
2. Before fitting, we need to remove the index column

In [41]:
```
x = housing.drop(["Id", "SalePrice"], axis=1)
y = housing["SalePrice"]

print(x[:2])
print(y[:2])
print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

```
   LotArea  OverallQual  1stFlrSF  GrLivArea  TotRmsAbvGrd  GarageCars  \
0     8450            7       856       1710             8           2
1     9600            6      1262       1262             6           2

   GarageArea
0         548
1         460
0    208500
1    181500
Name: SalePrice, dtype: int64
Input shape: (1460, 7)
Output shape: (1460,)
```

Linear Regression using sklearn for baseline

In [42]:
```
X = x
Y = y
```

In [43]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline


pipeline = Pipeline([
    ("scaling", StandardScaler()),
    ("regression", LinearRegression()),
])

pipeline.fit(X, Y)

y_predicted = pipeline.predict(X)
mse = mean_squared_error(Y, y_pred=y_predicted)

regression = pipeline.named_steps["regression"]
print(f"Intercept: {regression.intercept_}")
print(f"Coefficint: {np.round(regression.coef_)}")
```

```
Intercept: 180921.19589041095
Coefficint: [ 6163. 36532. 10921. 21661. -1785. 10169.  3988.]
```

Using Gradient Descent

In [44]:
```
import numpy as np

# STandardizing featuers before Gradient Descent

X_mean = X.mean(axis=0)
X_std = X.std(axis=0)

X_standardized = (X - X_mean ) / X_std

# Initialization

N, d = X_standardized.shape #Nmber of training samples, d = number of features
w = np.zeros(d)
b = 0.0

alpha = 0.01
epochs = 1000

# training loop

for _ in range(epochs):
    y_hat = X_standardized @ w + b
    error = y_hat - Y

    dw = (2 / N) * (X_standardized.T @ error)
    db = (2 / N) * np.sum(error)

    w = w - alpha * dw
    b = b - alpha * db
```

```
# evaluation

y_predicted = X_standardized @ w + b
mse = np.mean((Y - y_predicted) ** 2)

print("GD MSE:", mse)
print("GD intercept:", b)
print("GD coeff:\n", np.round(w, 3))
```

```
GD MSE: 1499583556.6556299
GD intercept: 180921.19558592647
GD coeff:
 LotArea          6192.815
OverallQual     36692.673
1stFlrSF        10972.833
GrLivArea       21132.665
TotRmsAbvGrd    -1365.205
GarageCars       9836.336
GarageArea       4314.647
dtype: float64
```

Gradient descent with convergence check and tolerance

In [46]:
```
import numpy as np

# STandardizing featuers before Gradient Descent

X_mean = X.mean(axis=0)
X_std = X.std(axis=0)

X_standardized = (X - X_mean ) / X_std

# Initialization

N, d = X_standardized.shape #Nmber of training samples, d = number of features
w = np.zeros(d)
b = 0.0

alpha = 0.01
max_itertions = 10000
tolerance = 1e-6

prev_loss = np.inf
converged = False

# training looop

for i in range(max_itertions):

    y_hat = X_standardized @ w + b
    error = y_hat - Y
```

```
    loss = np.mean(error**2)

    # convergence check
    if abs(prev_loss - loss) < tolerance:
        converged = True
        break

    prev_loss = loss

    dw = (2 / N) * (X_standardized.T @ error)
    db = (2 / N) * np.sum(error)

    w = w - alpha * dw
    b = b - alpha * db

iterations_used = i + 1

# reporting metrics

y_predicted = X_standardized @ w + b

mse = np.mean((Y - y_predicted ) ** 2)

ss_residual = np.sum((Y - y_predicted) ** 2)
ss_total = np.sum((Y - np.mean(Y))** 2)

r2 = 1 - ss_residual / ss_total
```

In [48]:
```
print("Gradient Descent Linear Regression Results")
print("----------------------------------------")
print(f"Learning rate used        : {alpha}")
print(f"MSE                       : {mse:.6f}")
print(f"R^2 score                 : {r2:.6f}")

if converged:
    print(f"Convergence status        : Converged")
    print(f"Iterations to convergence : {iterations_used}")
else:
    print(f"Convergence status        : Not converged")
    print(f"Iterations performed      : {iterations_used}")
```

```
Gradient Descent Linear Regression Results
----------------------------------------
Learning rate used        : 0.01
MSE                       : 1499498524.208768
R^2 score                 : 0.762241
Convergence status        : Converged
Iterations to convergence : 5370
```

Final trained model

In [49]:
```
# Denormalize the coefficient to the original scale

beta = w / X_std
beta_0 = b - np.sum((w * X_mean) / X_std)
```

```
print("Final Trained Linear Regression Model")
print("-----------------------------------")
print(f"Intercept (Beta_0): {beta_0:.4f}")

for i, coef in enumerate(beta, start=1):
    print(f"Beta_{i}: {coef:.4f}")
```

```
Final Trained Linear Regression Model
-----------------------------------
Intercept (Beta_0): -107805.4809
Beta_1: 0.6177
Beta_2: 26423.8401
Beta_3: 28.2590
Beta_4: 41.2357
Beta_5: -1098.8345
Beta_6: 13612.2358
Beta_7: 18.6587
```